

Today's work — you can see why a comment got flagged

Dyuthi • 2026-09-04 • written in my own words, not a formal report

The thing I had to ship today: someone who has never read my rules should be able to look at the review screen and understand why an item got flagged, on their own, without asking me. So that's what I built. Here it is in plain words.

1. Every comment now says what it broke, right in the list

Each row in the queue now carries small labels — **Too long**, **Generic phrase**, **No specific reference**, **Formula question**, **Near-identical to others**, and a few more. Plain English, no codes, no jargon. You don't have to open anything to see them.

Open a row and the same label is there again, but with the actual detail next to it — not just “Too long”, but “Too long for a comment: 514 characters (limit 400)”, and then whether that stops you approving it or just costs points.

2. The words come from one place, so they can't drift apart

This was the part of the ask I cared most about. It would have been easy to type the rule names into the screen by hand — and then next month the spec says one thing, the code says another, and the screen says a third. So I wrote the rule names down **once**, in the checking code, and the screen just reads them. Same word in the spec, same word in the code, same word on the chip. There is no second copy to fall out of step.

I also added a test that fails on purpose if anyone starts hard-coding a rule name back into the screen. Rules I can't enforce tend not to survive.

3. You can now filter and sort by the type of mistake

Above the list there's a row of buttons, one per problem, each with a count — “Generic phrase 14”, “No specific reference 9”, “Nothing flagged 5”. Click one and the list shows only those. There's also a sort switch: **Best match** (what it did before) or **Failure type**, which puts everything that broke the same rule together, worst first.

That's the part that saves real time. Fixing fourteen of the same mistake in one pass is one decision. Meeting the same mistake fourteen separate times, in date order, is fourteen.

The counts stay put when you filter — they're counted over the whole queue, not the filtered view, so you can always see what else is waiting and click back out.

4. The writing box counts as you type, and warns before the wall

When you edit a comment, the line underneath now says something like “268 / 400 characters · 3 sentences”. It has three states, not two:

- **Grey** — you're fine, carry on
- **Amber** — you've gone past 280 characters, which is what the rule asks for, or past 3 sentences. You can still send it. Fix it now and it costs you nothing.
- **Red** — past 400, which is the number that actually blocks. The Approve button goes grey and says so.

Why two numbers and not one: our written rule says 280, but the code has always blocked at 400. I didn't want the box to pretend 280 blocks when it doesn't — a counter that lies gets ignored within a week. So it shows you both: the target in amber, the wall in red.

Small thing I caught while testing: the counter said 515 and the warning underneath said 514, because the checker trims spaces off the end and the counter didn't. Fixed. A counter that disagrees with the rule it's counting against is worse than no counter.

5. Two rules that were written down but never shown — now visible, still not blocking

Two of my rules have never actually been checked by the code: “a comment is 1 to 3 sentences”, and the big one from the August audit — the “have you found any specific strategies?” ending that 43 out of 58 comments used.

I've made both of them **visible** today, and deliberately not made them block anything. They show up as grey “heads up” labels, they don't cost the comment any points, and they don't stop an approval. The reason is that the checker is shared with connection requests and direct messages, so tightening it changes what gets rejected on those too — that's Hemang's call, not mine to make alone. But there was no reason the operator couldn't see it in the meantime. Seeing it costs nothing and risks nothing.

6. I tested it on real comments, not made-up ones

I ran all 58 real comments from the August audit — the ones the bot actually wrote — through the new labels, and compared against the grades I gave them by hand back then:

- All **6** I graded **good** came back with **zero** labels. No false alarms on the good ones.
- All **11** I graded **fail** came back with at least one label. It caught every one.
- **45 of the 58** now carry a visible label — while the old quality score gave **54 of those same 58** a perfect 100 out of 100.

That last line is the whole point of today. The score was never wrong exactly — it just couldn't see the thing that was actually wrong. Now the screen says it out loud.

Then I opened the real app and used it

Loaded a queue of 30 real comments with 10 different problems mixed in and clicked through it myself:

- Filtered by “No specific reference” — got exactly those 9, counts stayed put
- Sorted by failure type — blocked ones first, then the repeated ones grouped together
- Opened a 515-character comment: counter went red, Approve went grey, three labels explained why
- Typed one up to 280 characters and watched it go amber, with Approve still available
- Zero errors in the browser console
- 264 automatic tests pass (24 of them new today), type-checker clean, production build clean

7. Checking the colours caught a real problem

Since these labels are now the main thing you read, I ran the readability math on them instead of trusting my eyes. Two of our existing colours failed badly when used as *text*: the green measured **2.36** and the red **3.40**, where **4.5** is the pass mark.

The cause is simple once you see it: those two colours were only ever picked to sit *behind* white text on a button. A colour dark enough to carry white text is far too dark to be text itself. So each one is now a pair — the button colour stays exactly as it was, and there's a second, lighter version used only for text. Green is now 6.72, red 4.63. I also added the amber, which passes everywhere I use it. All of it is written down in the contrast file with the numbers, so nobody has to redo the maths.

It fixed two older spots as a side effect too — the “Never contact” button label and the match-score badge were both quietly failing the same way.

8. Where it all lives, if you want to look

- **src/outreach/quality.py** — the one list of rule names, and the two new checks (sentences, formula question)
- **src/api/routes/outreach.py** — sends the labels to the screen, re-checked against the text as it stands now, so it stays true after an edit
- **frontend/src/pages/Approvals.tsx** — the labels, the filter row, the sort switch, the live counter

- **frontend/src/lib/guardrails.ts** — the display side only. No rule names in here, on purpose.
- **tests/test_guardrail_flags.py** — the 24 new tests
- **docs/CONTRAST_CHECK.md** — the colour numbers

9. What I have not done, so it's not a surprise later

- **I haven't sat Harshil in front of it yet.** That test is the real one and I can't mark my own homework on it. I built for it — plain words on the labels, the rule number tucked away as a hover, and a one-line explanation of what each severity means — but until I watch him use it and see where he pauses, it's my guess, not a result.
- **Three of the checks aren't in my written spec yet** — the pushy call-to-action one, the generic-opener one, and the repeated-comment one. Rather than dress them up as spec rules, the screen says “not in the spec yet” next to them. That's a v2 job for the spec, not a rename.
- **The 1-to-3-sentence rule and the formula-question rule still don't block anything.** That needs Hemang and a booked slot, because it changes connection requests and messages too.
- **Nothing is committed or pushed.** It's all sitting in the working copy, tested and running. Say the word and I'll push it.

The honest summary: the checker always knew why it docked a comment, it just never said so in a way you could group, count, or act on. Now it does — and the two rules it still can't enforce, it at least admits to instead of hiding.